

API SECURITY RISK ANALYSIS REPORT

Name: Harshitha Vardhini Mademu

Date: 18th Apr, 2026

Introduction:

Application Programming Interfaces (APIs) play a critical role in modern software systems by enabling communication between different applications, services, and platforms. APIs are widely used in web applications, mobile apps, and Software-as-a-Service (SaaS) platforms to exchange data and perform operations efficiently.

However, due to their widespread use, APIs have become a major target for attackers. Insecure APIs can expose sensitive data, allow unauthorized access, and enable attackers to manipulate system functionality. Common API security issues include lack of authentication, broken authorization, excessive data exposure, and insufficient input validation.

This report presents a structured API security risk analysis conducted on a public test API. The assessment was performed using a controlled and read-only approach, focusing on identifying potential risks without exploiting vulnerabilities. The goal of this analysis is to evaluate how the API handles authentication, authorization, data exposure, and request validation, and to provide practical recommendations to improve security.

The testing was carried out using **Postman** and Browser Developer Tools in a Kali Linux environment, simulating a real-world API security assessment workflow.

Objective:

The objectives of this assessment are:

- To analyze a public API for common security risks
- To evaluate authentication and authorization mechanisms
- To identify issues related to data exposure and input validation
- To classify risks based on severity and business impact
- To suggest appropriate remediation strategies

Scope:

This assessment is limited to:

- Public API testing using safe methods
- Read-only operations (GET requests only)
- No exploitation or harmful testing

Focus Areas:

- Authentication
- Authorization
- Data exposure
- Rate limiting
- Input validation

API Tested:

- **API Name:** JSONPlaceholder
- **Base URL:** <https://jsonplaceholder.typicode.com>
- **Type:** Public testing API

Endpoints Tested:

- /users
- /users/1
- /users/2
- /posts?userId=abc

Methodology:

The following steps were followed:

1. API documentation was reviewed to understand endpoints
2. Requests were sent using Postman
3. Responses were analyzed for exposed data
4. Browser Developer Tools were used to inspect headers
5. Multiple requests were sent to test rate limiting
6. Input parameters were modified to test validation

All testing was conducted ethically without exploiting any vulnerabilities.

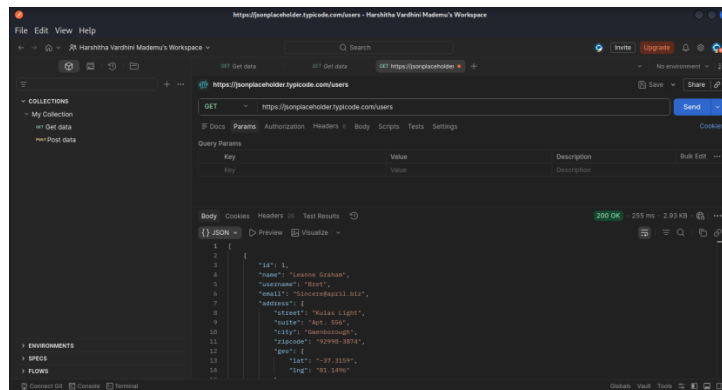
FINDING 1: Unauthenticated Access

Severity: High

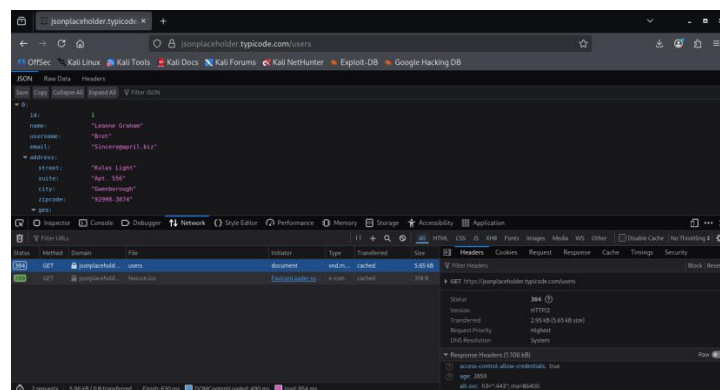
Description:

The API does not require authentication to access its endpoints.

Evidence:



Users screenshot



BrowserDevTools headers screenshot

Business Impact:

Unauthorized users can access sensitive data.

Remediation:

- Implement JWT/OAuth authentication
- Require tokens for API access

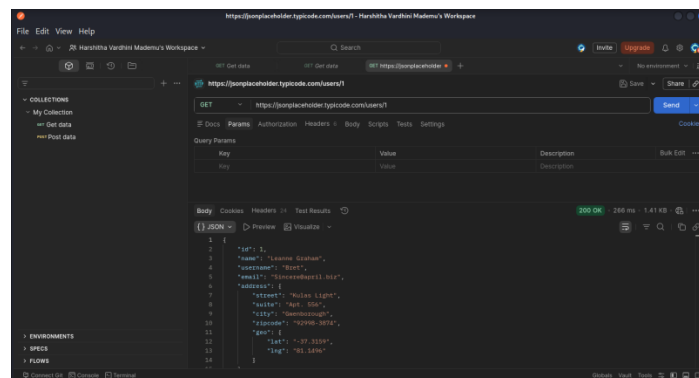
FINDING 2: Broken Authorization

Severity: High

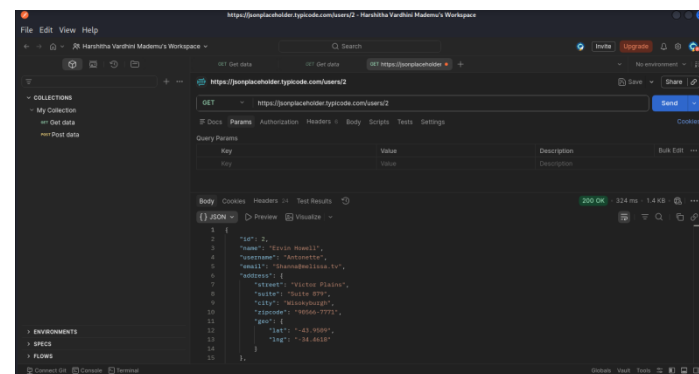
Description:

Users can access other users' data by modifying the ID.

Evidence:



user1 screenshot



users2 screenshot

Business Impact:

Leads to data leakage and privacy issues.

Remediation:

- Implement RBAC
- Validate user identity

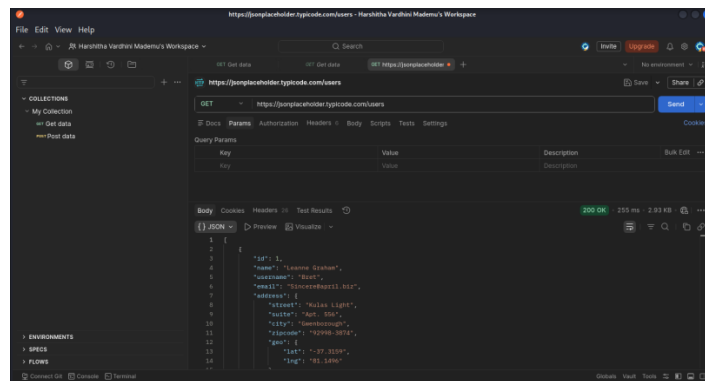
FINDING 3: Excessive Data Exposure

Severity: Medium

Description:

API exposes unnecessary user details.

Evidence:



users response screenshot

Business Impact:

Sensitive data exposure.

Remediation:

- Limit response fields
- Apply data filtering

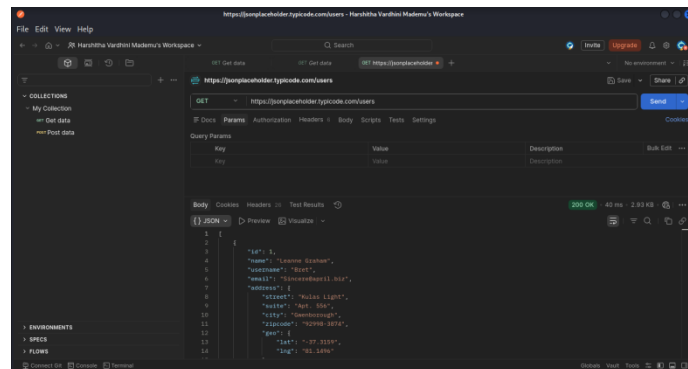
FINDING 4: Lack of Rate Limiting

Severity: Medium

Description:

Unlimited requests allowed.

Evidence:



Insert repeated request screenshot

Business Impact:

API abuse and overload risk.

Remediation:

- Implement rate limiting
- Use throttling

FINDING 5: Weak Input Validation

Severity: Low

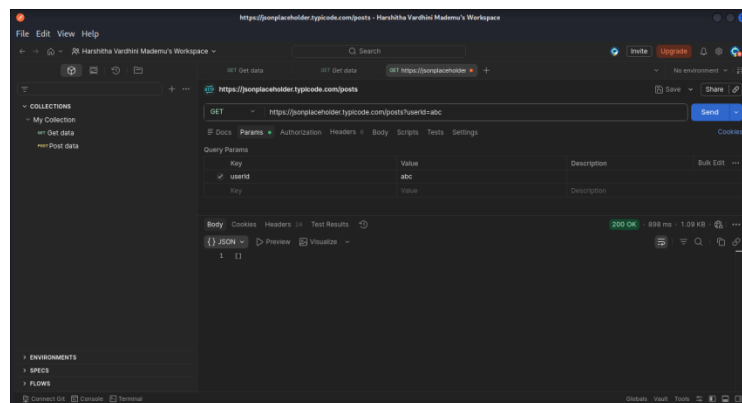
Description:

Invalid inputs are accepted.

Tested Endpoint:

/posts?userId=abc

Evidence:



Insert invalid input screenshot

Business Impact:

May lead to vulnerabilities in real systems.

Remediation:

- Validate inputs
- Reject invalid data

RISK SUMMARY TABLE:

Risk	Severity	Business Impact
Unauthenticated Access	High	Data exposure
Broken Authorization	High	Data leakage
Excessive Data Exposure	Medium	Privacy risk
No Rate Limiting	Medium	Abuse risk
Weak Input Validation	Low	Security weakness

RECOMMENDATIONS:

- Implement authentication (JWT/OAuth)
- Enforce Role-Based Access Control
- Limit data exposure
- Apply rate limiting
- Validate inputs
- Monitor API usage

CONCLUSION:

The API security assessment identified multiple vulnerabilities, especially in authentication and authorization. While the API is designed for testing purposes, similar issues in real-world applications can lead to serious security risks.

Proper implementation of security controls can significantly reduce these risks and improve overall API security.